You said:

```
PS C:\Users\user\Downloads\37_climber-main> dir


    Directory: C:\Users\user\Downloads\37_climber-main


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/16/2026   9:29 AM                37_climber-main


PS C:\Users\user\Downloads\37_climber-main>
```

ChatGPT said:

You have one more folder inside the current folder. That's why `game.py` wasn't found. Let's enter the inner folder.

Step 1 — Open the correct folder

Copy and run this command in PowerShell:

```
cd .\37_climber-main
```

Step 2 — Check its contents

```
dir
```

You should now see the assignment files.

Step 3 — Find the Python file

Run:

```
Get-ChildItem -Recurse -Filter *.py | Select-Object FullName
```

If you see `game.py` in the listing, run:

```
py game.py
```

If the Python file has a different name, don't run that last command yet.

Send me the output of `dir` after Step 1. I'll guide you through the exact next step.

You said:

```
PS C:\Users\user\Downloads\37_climber-main> cd .\37_climber-main
PS C:\Users\user\Downloads\37_climber-main\37_climber-main> dir


    Directory: C:\Users\user\Downloads\37_climber-main\37_climber-
main


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/16/2026   9:29 AM             13 .gitignore
-a----        10/16/2026   9:29 AM           7635 game.py
-a----        10/16/2026   9:29 AM           4712 README.md


PS C:\Users\user\Downloads\37_climber-main\37_climber-main>
```

ChatGPT said:

Perfect! ✅ You're now in the correct folder, and `game.py` is present.

Let's continue step by step.

Step 1 — Install Pygame

Run this command:

```
py -m pip install pygame
```

Wait until the installation finishes. If it says Pygame is already installed, that's fine.

Step 2 — Run the original game

Now run:

```
py game.py
```

- If the game window opens, play for a few seconds.
- If you see an error in PowerShell, copy and send me the complete error.
- If the game opens successfully, **record your original 10-second gameplay video before editing any code**.

Step 3 — Open the code in VS Code

From the same folder, run:

```
code .
```

If that command isn't recognized, open VS Code manually and select **File** → **Open Folder**, then choose:

```
C:\Users\user\Downloads\37_climber-main\37_climber-main
```

Open `game.py`.

Do these steps in order and tell me what happens when you run `py game.py`**.** Then I'll help you implement and test each assignment task.

You said:

opened game took screenshot next

ChatGPT said:

Great! Now that you've opened the original game and taken a screenshot, let's complete the assignment in the correct order.

Step 1 — Save your original gameplay evidence

Your assignment requires a **10-second video before changes**, not just a screenshot.

1. Press `Win + Shift + R` to open Windows screen recording, if available.
2. Select the game window and record about 10 seconds of gameplay.
3. Save the video as `before_changes.mp4`.

If that shortcut isn't available on your computer, use Snipping Tool's video-recording option.

Step 2 — Open `game.py` in VS Code

In your PowerShell terminal, run:

```
code .
```

Open `game.py` from the Explorer panel.

Step 3 — Find the four tasks

Press `Ctrl + F` and search for each of these one at a time:

- `self.height = current_height`
- `def platform_color`
- `def moving_platform_speed`
- `def on_coin_collected`

These are the areas we need to inspect and modify.

Before changing anything: send me the code around these four sections (you can copy and paste it here, or send screenshots from VS Code). I'll help you make the changes that match your actual starter code and avoid introducing errors.

You said:

```
import random
import pygame

WIDTH, HEIGHT = 480, 640
FPS = 60
BG = (20, 20, 40)
GRAVITY, JUMP_SPEED, MOVE_SPEED = 0.5, -13, 4
PLAYER_W, PLAYER_H = 36, 36
PLATFORM_H = 14
```

```
COIN_R = 7
LIVES_START = 3


def platform_color(index, total):
    """Return an (r, g, b) colour override for the platform at this index (0
is the ground), or None for the default green."""
    pass


def moving_platform_speed(index, total):
    """Return a horizontal oscillation speed in pixels/frame for the
platform at this index, or None/0 to keep it static."""
    pass


def on_coin_collected(coin, score):
    """Called the instant the player collects a coin, after its value has
been added to the score. Add a sound or sparkle here."""
    pass


class Platform:
    def __init__(self, index, total, x, y, w=120, movable=True):
        self.rect = pygame.Rect(x, y, w, PLATFORM_H)
        self.speed = (moving_platform_speed(index, total) or 0) if
movable else 0
        self.bounds = (max(0, x - 60), min(WIDTH - w, x + 60))
        self.color = platform_color(index, total) or (100, 180, 100)

    def update(self):
        if not self.speed:
            return 0
        dx = self.speed
        if self.rect.x <= self.bounds[0] or self.rect.x >= self.bounds[1]:
            self.speed = -self.speed
            dx = self.speed
        self.rect.x = max(self.bounds[0], min(self.bounds[1], self.rect.x +
dx))
        return dx

    def draw(self, screen, cam_y):
```

```
        pygame.draw.rect(screen, self.color, self.rect.move(0, -cam_y),
border_radius=4)


class Coin:
    def __init__(self, x, y):
        self.pos = pygame.Vector2(x, y)
        self.taken = False

    def draw(self, screen, cam_y):
        pygame.draw.circle(screen, (250, 210, 60), (self.pos.x, self.pos.y -
cam_y), COIN_R)


def generate_platforms(num, start_y, width):
    platforms = [Platform(0, num, 0, start_y, width, movable=False)]  #
ground never moves
    y = start_y - 100
    for i in range(1, num + 1):
        x = random.randint(20, width - 140)
        w = random.randint(80, 160)
        platforms.append(Platform(i, num, x, y, w))
        y -= random.randint(70, 120)
    return platforms


def spawn_coins(platforms):
    coins = []
    for plat in platforms[1:]:
        if random.random() < 0.4:
            coins.append(Coin(plat.rect.centerx, plat.rect.top - 14))
    return coins


class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, PLAYER_W, PLAYER_H)
        self.vel_y = 0.0
        self.vel_x = 0
        self.on_ground = False
        self.standing_on = None
        self.color = (60, 120, 200)
```

```
    def move(self, keys):
        self.vel_x = 0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.vel_x = -MOVE_SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.vel_x = MOVE_SPEED
        if (keys[pygame.K_SPACE] or keys[pygame.K_w] or
keys[pygame.K_UP]) and self.on_ground:
            self.vel_y = JUMP_SPEED
            self.on_ground = False

    def update(self, platforms):
        self.vel_y = min(self.vel_y + GRAVITY, 12)
        previous_bottom = self.rect.bottom
        self.rect.x = max(0, min(WIDTH - self.rect.width, self.rect.x +
self.vel_x))
        self.rect.y += int(self.vel_y)
        self.on_ground = False
        self.standing_on = None
        if self.vel_y >= 0:
            for plat in platforms:
                overlaps = self.rect.right > plat.rect.left and self.rect.left <
plat.rect.right
                if overlaps and previous_bottom <= plat.rect.top + 1 and
self.rect.bottom >= plat.rect.top:
                    self.rect.bottom = plat.rect.top
                    self.vel_y = 0
                    self.on_ground = True
                    self.standing_on = plat
                    break

    def draw(self, screen, cam_y):
        draw_rect = self.rect.move(0, -cam_y)
        pygame.draw.rect(screen, self.color, draw_rect, border_radius=6)
        pygame.draw.circle(screen, (255, 220, 180), (draw_rect.centerx,
draw_rect.top + 10), 7)


class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
```

```
        self.big_font = pygame.font.Font(None, 42)
        self.reset()

    def reset(self):
        self.platforms = generate_platforms(60, HEIGHT - 40, WIDTH)
        self.coins = spawn_coins(self.platforms)
        self.player = Player(WIDTH // 2 - PLAYER_W // 2, HEIGHT - 100)
        self.cam_y = 0
        self.height = 0
        self.coin_score = 0
        self.lives = LIVES_START
        self.last_safe = pygame.Vector2(self.player.rect.x, self.player.rect.y)
        self.state = "play"
        self.top_y = self.platforms[-1].rect.y

    def score(self):
        return int(self.height) + self.coin_score

    def update(self, keys):
        if self.state != "play":
            return
        self.player.move(keys)
        for plat in self.platforms:
            dx = plat.update()
            if dx and self.player.standing_on is plat:
                self.player.rect.x += dx
        self.player.update(self.platforms)

        target_cam = self.player.rect.centery - HEIGHT // 2
        if target_cam < self.cam_y:
            self.cam_y = target_cam

        current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
        self.height = current_height

        if self.player.on_ground:
            self.last_safe = pygame.Vector2(self.player.rect.x,
self.player.rect.y)

        for coin in self.coins:
            if not coin.taken and self.player.rect.collidepoint(coin.pos):
                coin.taken = True
```

```
                self.coin_score += 50
                on_coin_collected(coin, self.score())
        self.coins = [c for c in self.coins if not c.taken]

        if self.player.rect.top - self.cam_y > HEIGHT + 50:
            self.lives -= 1
            if self.lives <= 0:
                self.state = "lose"
            else:
                self.player.rect.x, self.player.rect.y = int(self.last_safe.x),
int(self.last_safe.y)
                self.player.vel_y = 0

        if self.player.rect.y <= self.top_y:
            self.state = "win"

    def draw(self, screen):
        screen.fill(BG)
        for plat in self.platforms:
            plat.draw(screen, self.cam_y)
        for coin in self.coins:
            coin.draw(screen, self.cam_y)
        self.player.draw(screen, self.cam_y)

        hud = self.font.render(f"Height: {self.height}m  Coins:
{self.coin_score // 50}  Lives: {self.lives}", True, (200, 200, 200))
        screen.blit(hud, (10, 10))

        if self.state != "play":
            text = "YOU REACHED THE TOP!" if self.state == "win" else
"YOU FELL!"
            color = (80, 220, 80) if self.state == "win" else (220, 60, 60)
            msg = self.big_font.render(text, True, color)
            sub = self.font.render("Press R to Restart", True, (180, 180,
180))
            screen.blit(msg, msg.get_rect(center=(WIDTH // 2, HEIGHT // 2
- 20)))
            screen.blit(sub, sub.get_rect(center=(WIDTH // 2, HEIGHT // 2
+ 30)))


def main():
```

```
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Climber")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
                game.reset()
        game.update(pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()
        clock.tick(FPS)
    pygame.quit()


if __name__ == "__main__":
    main()
```

ChatGPT said:

Perfect. This is the exact starter code. We can now implement all four tasks in your actual `game.py`.

We'll make the changes carefully so you can run and demonstrate each feature.

Step 1 — Fix the height score

In VS Code, find this line inside `Game.update()`:

```
self.height = current_height
```

Replace it with:

```
self.height = max(self.height, current_height)
```

This ensures the displayed height keeps the highest value reached, even when the player falls.

Step 2 — Add platform colours

Find the first function near the top:

```
def platform_color(index, total):
    """Return an (r, g, b) colour override for the platform at this index (0 is the ground), or None for the default green."""
    pass
```

Replace the entire function with:

```
def platform_color(index, total):
    """Return a gradual colour transition as platforms get higher."""
    progress = index / max(1, total)

    red = int(100 + 155 * progress)
    green = int(180 - 100 * progress)
    blue = int(100 - 40 * progress)

    return (red, green, blue)
```

The platforms will transition from green at the bottom to orange-red at the top.

Step 3 — Add moving platforms

Find:

```
def moving_platform_speed(index, total):
    """Return a horizontal oscillation speed in pixels/frame for the platform at this index, or None/0 to keep it static."""
    pass
```

Replace it with:

```
def moving_platform_speed(index, total):
    """Make every third platform move horizontally."""
```

```
    if index > 0 and index % 3 == 0:
        return 1.5 if index % 2 == 0 else -1.5

    return 0
```

The existing `Platform.update()` already moves platforms and reverses direction at their boundaries, so no changes to that class are required.

Step 4 — Add the coin-collection sparkle

We'll make a short-lived yellow sparkle appear when a coin is collected.

4A. Create the sparkle list

Find:

```
LIVES_START = 3
```

Immediately below it, add:

```
COIN_SPARKLES = []
```

4B. Implement the coin callback

Replace the existing `on_coin_collected()` function with:

```
def on_coin_collected(coin, score):
    """Create a sparkle when a coin is collected."""
    COIN_SPARKLES.append({
        "x": coin.pos.x,
        "y": coin.pos.y,
        "life": 15,
    })
```

4C. Update the sparkle lifetime

Inside `Game.update()`, find this section:

```
self.coins = [c for c in self.coins if not c.taken]
```

Immediately below it, add the following code, keeping the indentation consistent with the other statements inside `update()`:

```
        for sparkle in COIN_SPARKLES[:]:
            sparkle["life"] -= 1

            if sparkle["life"] <= 0:
                COIN_SPARKLES.remove(sparkle)
```

This reduces the lifespan every frame and removes expired sparkles.

4D. Draw the sparkle

Inside `Game.draw()`, find:

```
for coin in self.coins:
    coin.draw(screen, self.cam_y)
```

Immediately below that loop, add:

```
        for sparkle in COIN_SPARKLES:
            x = int(sparkle["x"])
            y = int(sparkle["y"] - self.cam_y)
            life = sparkle["life"]

            color = (255, 255, 180)
            radius = max(1, life // 4)

            pygame.draw.circle(
                screen, color, (x, y), radius, 2
            )
```

Keep the existing player drawing line below the new code:

```
self.player.draw(screen, self.cam_y)
```

4E. Clear sparkles on restart

Inside `Game.reset()`, add this line at the beginning of the method:

```
        COIN_SPARKLES.clear()
```

Your method should now begin like this:

```
def reset(self):
    COIN_SPARKLES.clear()
    self.platforms = generate_platforms(60, HEIGHT - 40, WIDTH)
```

Step 5 — Save and run the game

Save the file using `Ctrl + S`.

In PowerShell, run:

```
py game.py
```

If the game is already open, close it first and run the command again.

Step 6 — Test your completed assignment

Use this checklist as you test:

- [ ] **Height score**
  Climb up and fall. The height display should never decrease.
- [ ] **Platform colours**
  Compare the colours of lower and higher platforms.
- [ ] **Moving platforms**
  Look for platforms moving horizontally and reversing direction at their limits.
- [ ] **Coin effect**
  Collect a coin. A small yellow ring should briefly appear.

- [ ] **Restart**
  Press R after winning or losing. The game should restart without leftover sparkles.

0 of 5 features tested.

Your next action: Make Steps 1–4, save, and run `py game.py`. If you encounter any error, send me the full terminal message. Once the game works, we'll finish your after-changes video and submission checklist.

You said:

give the full code with all the requirements

ChatGPT said:

Below is the complete updated `game.py` for your Climber assignment. It includes the three requested improvements:

1. **Height score fix** — your highest height stays recorded even if you fall.
2. **Platform colour progression** — platforms change colour as you climb higher.
3. **Moving platforms** — every third platform moves horizontally.
4. **Coin collection sparkles** — collecting a coin triggers a short visual effect.

It keeps the original Pygame game structure, including coins, lives, camera scrolling, win/lose states, and restart behavior.

Open `game.py` in your folder:

```
C:\Users\user\Downloads\37_climber-main\37_climber-main
```

Replace the entire contents with the code below, save it, and run `py game.py`.

Complete updated `game.py`

```
import random
import pygame

WIDTH, HEIGHT = 480, 640
FPS = 60
BG = (20, 20, 40)
```

```
GRAVITY, JUMP_SPEED, MOVE_SPEED = 0.5, -13, 4
PLAYER_W, PLAYER_H = 36, 36
PLATFORM_H = 14
COIN_R = 7
LIVES_START = 3

# Stores temporary coin collection sparkle effects.
COIN_SPARKLES = []


def platform_color(index, total):
    """Gradually change platform colours as the player climbs."""
    progress = index / max(1, total)

    red = int(100 + 155 * progress)
    green = int(180 - 100 * progress)
    blue = int(100 - 40 * progress)

    return (red, green, blue)


def moving_platform_speed(index, total):
    """Make every third platform move horizontally."""
    if index > 0 and index % 3 == 0:
        return 1.5 if index % 2 == 0 else -1.5

    return 0


def on_coin_collected(coin, score):
    """Create a sparkle effect when a coin is collected."""
    COIN_SPARKLES.append({
        "x": coin.pos.x,
        "y": coin.pos.y,
        "life": 15,
    })


class Platform:
    def __init__(self, index, total, x, y, w=120, movable=True):
        self.rect = pygame.Rect(x, y, w, PLATFORM_H)
```

```
        self.speed = (
            moving_platform_speed(index, total) or 0
        ) if movable else 0

        self.bounds = (
            max(0, x - 60),
            min(WIDTH - w, x + 60)
        )

        self.color = platform_color(index, total) or (
            100, 180, 100
        )

    def update(self):
        if not self.speed:
            return 0

        dx = self.speed

        if (
            self.rect.x <= self.bounds[0]
            or self.rect.x >= self.bounds[1]
        ):
            self.speed = -self.speed
            dx = self.speed

        self.rect.x = max(
            self.bounds[0],
            min(self.bounds[1], self.rect.x + dx)
        )

        return dx

    def draw(self, screen, cam_y):
        pygame.draw.rect(
            screen,
            self.color,
            self.rect.move(0, -cam_y),
            border_radius=4
        )
```

```
class Coin:
    def __init__(self, x, y):
        self.pos = pygame.Vector2(x, y)
        self.taken = False

    def draw(self, screen, cam_y):
        pygame.draw.circle(
            screen,
            (250, 210, 60),
            (int(self.pos.x), int(self.pos.y - cam_y)),
            COIN_R
        )


def generate_platforms(num, start_y, width):
    # The ground platform never moves.
    platforms = [
        Platform(0, num, 0, start_y, width, movable=False)
    ]

    y = start_y - 100

    for i in range(1, num + 1):
        x = random.randint(20, width - 140)
        w = random.randint(80, 160)

        platforms.append(Platform(i, num, x, y, w))

        y -= random.randint(70, 120)

    return platforms


def spawn_coins(platforms):
    coins = []

    for plat in platforms[1:]:
        if random.random() < 0.4:
            coins.append(
                Coin(plat.rect.centerx, plat.rect.top - 14)
            )
```

```
    return coins


class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, PLAYER_W, PLAYER_H)

        self.vel_y = 0.0
        self.vel_x = 0

        self.on_ground = False
        self.standing_on = None

        self.color = (60, 120, 200)

    def move(self, keys):
        self.vel_x = 0

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.vel_x = -MOVE_SPEED

        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.vel_x = MOVE_SPEED

        if (
            keys[pygame.K_SPACE]
            or keys[pygame.K_w]
            or keys[pygame.K_UP]
        ) and self.on_ground:
            self.vel_y = JUMP_SPEED
            self.on_ground = False

    def update(self, platforms):
        self.vel_y = min(self.vel_y + GRAVITY, 12)

        previous_bottom = self.rect.bottom

        self.rect.x = max(
            0,
            min(
                WIDTH - self.rect.width,
```

```
                self.rect.x + self.vel_x
            )
        )

        self.rect.y += int(self.vel_y)

        self.on_ground = False
        self.standing_on = None

        # Land only when falling onto a platform from above.
        if self.vel_y >= 0:
            for plat in platforms:
                overlaps = (
                    self.rect.right > plat.rect.left
                    and self.rect.left < plat.rect.right
                )

                if (
                    overlaps
                    and previous_bottom <= plat.rect.top + 1
                    and self.rect.bottom >= plat.rect.top
                ):
                    self.rect.bottom = plat.rect.top
                    self.vel_y = 0
                    self.on_ground = True
                    self.standing_on = plat
                    break

    def draw(self, screen, cam_y):
        draw_rect = self.rect.move(0, -cam_y)

        pygame.draw.rect(
            screen,
            self.color,
            draw_rect,
            border_radius=6
        )

        pygame.draw.circle(
            screen,
            (255, 220, 180),
            (draw_rect.centerx, draw_rect.top + 10),
```

```
            7
        )


class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.big_font = pygame.font.Font(None, 42)
        self.reset()

    def reset(self):
        # Clear sparkles from the previous run.
        COIN_SPARKLES.clear()

        self.platforms = generate_platforms(
            60, HEIGHT - 40, WIDTH
        )

        self.coins = spawn_coins(self.platforms)

        self.player = Player(
            WIDTH // 2 - PLAYER_W // 2,
            HEIGHT - 100
        )

        self.cam_y = 0
        self.height = 0
        self.coin_score = 0
        self.lives = LIVES_START

        self.last_safe = pygame.Vector2(
            self.player.rect.x,
            self.player.rect.y
        )

        self.state = "play"
        self.top_y = self.platforms[-1].rect.y

    def score(self):
        return int(self.height) + self.coin_score

    def update(self, keys):
```

```
        if self.state != "play":
            return

        self.player.move(keys)

        # Update moving platforms and carry the player with them.
        for plat in self.platforms:
            dx = plat.update()

            if dx and self.player.standing_on is plat:
                self.player.rect.x += dx

        self.player.update(self.platforms)

        # Scroll the camera upward, but never downward.
        target_cam = (
            self.player.rect.centery - HEIGHT // 2
        )

        if target_cam < self.cam_y:
            self.cam_y = target_cam

        # Keep the highest height reached during this run.
        current_height = max(
            0,
            (HEIGHT - 40 - self.player.rect.y) // 10
        )

        self.height = max(self.height, current_height)

        if self.player.on_ground:
            self.last_safe = pygame.Vector2(
                self.player.rect.x,
                self.player.rect.y
            )

        # Collect coins and trigger their sparkle effect.
        for coin in self.coins:
            if (
                not coin.taken
                and self.player.rect.collidepoint(coin.pos)
            ):
```

```
                coin.taken = True
                self.coin_score += 50

                on_coin_collected(coin, self.score())

        self.coins = [
            coin for coin in self.coins if not coin.taken
        ]

        # Update sparkle lifetimes.
        for sparkle in COIN_SPARKLES[:]:
            sparkle["life"] -= 1

            if sparkle["life"] <= 0:
                COIN_SPARKLES.remove(sparkle)

        # Lose a life if the player falls below the screen.
        if self.player.rect.top - self.cam_y > HEIGHT + 50:
            self.lives -= 1

            if self.lives <= 0:
                self.state = "lose"
            else:
                self.player.rect.x = int(self.last_safe.x)
                self.player.rect.y = int(self.last_safe.y)
                self.player.vel_y = 0

        # Win when the player reaches the top of the level.
        if self.player.rect.y <= self.top_y:
            self.state = "win"

    def draw(self, screen):
        screen.fill(BG)

        for plat in self.platforms:
            plat.draw(screen, self.cam_y)

        for coin in self.coins:
            coin.draw(screen, self.cam_y)

        # Draw the temporary coin collection sparkles.
        for sparkle in COIN_SPARKLES:
```

```
            x = int(sparkle["x"])
            y = int(sparkle["y"] - self.cam_y)
            life = sparkle["life"]

            color = (255, 255, 180)
            radius = max(1, life // 4)

            pygame.draw.circle(
                screen,
                color,
                (x, y),
                radius,
                2
            )

        self.player.draw(screen, self.cam_y)

        hud = self.font.render(
            f"Height: {self.height}m  "
            f"Coins: {self.coin_score // 50}  "
            f"Lives: {self.lives}",
            True,
            (200, 200, 200)
        )

        screen.blit(hud, (10, 10))

        if self.state != "play":
            text = (
                "YOU REACHED THE TOP!"
                if self.state == "win"
                else "YOU FELL!"
            )

            color = (
                (80, 220, 80)
                if self.state == "win"
                else (220, 60, 60)
            )

            msg = self.big_font.render(text, True, color)
```

```
            sub = self.font.render(
                "Press R to Restart",
                True,
                (180, 180, 180)
            )

            screen.blit(
                msg,
                msg.get_rect(
                    center=(WIDTH // 2, HEIGHT // 2 - 20)
                )
            )

            screen.blit(
                sub,
                sub.get_rect(
                    center=(WIDTH // 2, HEIGHT // 2 + 30)
                )
            )


def main():
    pygame.init()

    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Climber")

    clock = pygame.time.Clock()
    game = Game()

    running = True

    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

            elif (
                event.type == pygame.KEYDOWN
                and event.key == pygame.K_r
            ):
                game.reset()
```

```
        game.update(pygame.key.get_pressed())
        game.draw(screen)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()
```

Run and verify it

1. Save the file as `game.py` — make sure it doesn't become `game.py.txt`.
2. Open Command Prompt in your project folder.
3. Run:

```
py game.py
```

4. Use **A/Left** and **D/Right** to move, and **Space/W/Up** to jump. Press **R** to restart.

Check that the platforms have different colours, some platforms move horizontally, the height value never decreases during a run, and collecting coins produces a sparkle effect.

Remember your submission requirements: record a 10-second video before your changes (if you haven't already), a 10-second video after your changes, and include the ChatGPT conversation link with the complete chat history.